EE128 Project: Timer & Interrupt Implementation

Wiring:
Keypad Rows: PTB9, PTC2, PTC3, PTC4
Keypad Cols: PTA1, PTB23, PTA2, PTC0
LEDs: PTD0 (Locked), PTD1 (Unlocked), PTD2 (Error/Status) -> ACTIVE HIGH
Buzzer: PTD3 (Passive) -> FTM3_CH3
LCD: RS->PTC5, E->PTC1, D4-D7->PTD4-PTD7
Servo: PTB18 (FTM2_CH0)
Bluetooth: UART4 (PTE24/PTE25)

1. Timer Implementations (3 Hardware, 1 Software)

We utilized three distinct hardware timer modules to perform three unique types of timing tasks, demonstrating the full spectrum of timer applications discussed in class.

- PIT0 (Programmable Interval Timer):

  Type: Hardware Interval Timer.

  Function: Generates a precise 10ms heartbeat.

  Application: Acts as the central system clock. It drives the Auto-lock countdown (5s/2s), the Lockout countdown (30s), and drives the Software RTC logic.

  Lecture Connection: "Periodic Interrupt Generation" and "Time Delay Creation".

- FTM2 (FlexTimer Module 2):

  Type: PWM (Pulse Width Modulation).

  Function: Generates a 50Hz signal with a variable duty cycle (1ms to 2ms).

  Application: Controls the Servo Motor angle (0° vs 90°) entirely in hardware without blocking the CPU.

  Lecture Connection: "Pulse Width Modulation" and "Output Compare".

- FTM3 (FlexTimer Module 3):

  Type: PWM (Pulse Width Modulation).

Function: Generates variable frequency signals (e.g., 523Hz, 784Hz) with a fixed 50% duty cycle.

Application: Drives the Passive Buzzer to synthesize musical "Jingles" and audio feedback.

Lecture Connection: "Digital Waveform Generation" (Audio).

- Software RTC (Real-Time Clock):

  Type: Software Timer Logic.

  Function: Aggregates the 10ms PIT ticks into Seconds, Minutes, and Hours.

  Application: Enables Night Mode by tracking "Wall Clock" time to determine when to switch system behavior.

  Lecture Connection: "Time-of-Day Tracking".

---

2. Interrupt Implementations (3 Types)

The project features a robust Event-Driven Architecture. We utilized three distinct categories of interrupts to handle asynchronous events.

- GPIO Interrupts (External Interrupts):

  Source: PORTB and PORTC.

  Trigger: Falling Edge detection on keypad rows.

  Function: Wakes the CPU only when a button is physically pressed to scan the keypad matrix. This eliminates the need for inefficient polling in the main loop.

  Lecture Connection: "I/O Handling with Interrupts".

- Timer Interrupts (Periodic Interrupts):

  Source: PIT0.

  Trigger: Timer Overflow (every 10ms).

  Function: Updates system clocks, decrements countdown variables, and checks state transitions in the background.

  Lecture Connection: "Timer Overflow Interrupt".

- UART Interrupts (Communication Interrupts):

    Source: UART4 (Bluetooth Module).

    Trigger: Receiver Full (RDRF).

    Function: Triggers immediately when the Bluetooth module receives a character ('U', 'L', or 'T') from a remote device. The CPU reads the command into a buffer for processing.

    Lecture Connection: "Vector Table" (UART is a distinct vector from GPIO or Timer).

---

3. Detailed Feature Breakdown

A. Night Mode (Time-of-Day Logic)

- Feature: The system automatically switches behavior (Silent Mode + Fast 2s Auto-lock) between 20:00 and 06:00.
- Implementation: I implemented a Software RTC driven by the PIT0 Hardware Timer. The PIT0 ISR increments the time counters and updates the is_night_mode flag in the background, keeping the main loop free for UI tasks.

B. Lockout Mode (Timeout Logic)

- Feature: After 3 failed PIN attempts, the keypad is disabled for 30 seconds.
- Implementation: Instead of blocking the CPU with delay_ms(30000), I set a variable lockout_ticks = 3000. The PIT0 ISR decrements this counter every 10ms. This allows the LCD to update the countdown dynamically while the input remains blocked.

C. Permanent Unlock (State Control)

- Feature: Pressing 'A' disables the auto-lock timer entirely.
- Implementation: This demonstrates Timer State Control. The software explicitly disables the counting mechanism by setting autolock_ticks = -1. The ISR checks this condition before decrementing, effectively "pausing" the timer logic.

---

4. Demonstration Script

Step 1: The "Normal" Demo (Standard Operation)

- Action: Reset the board. The LCD says LOCKED 12:00.
- Action: Type 1 2 3 4 #.

- Result: Servo moves to 90 degrees, "Success" jingle plays, Green LED turns ON.
- Observation: "The door is unlocked. The auto-lock timer counts down from 5 seconds."
- Result: After 5s, Servo moves back to 0 degrees, "Lock" jingle plays, Red LED turns ON.

Step 2: Permanent Unlock (Hot Key 'A')

- Action: Unlock the door (1 2 3 4 #).
- Action: Press 'A'.
- Observation: LCD shows PERM UNLOCKED. The timer disappears.
- Action: Wait >5 seconds to prove it does not lock.
- Action: Press '*' to manually lock and exit mode.

Step 3: Change PIN (Hot Key 'C')

- Action: Unlock the door (1 2 3 4 #).
- Action: Press 'C'. Enter Old PIN (1 2 3 4 #). Enter New PIN (0 0 0 0 #).
- Proof: Lock the door. Try 1 2 3 4 # (Error). Unlock with 0 0 0 0 # (Success).

Step 4: Night Mode (Hot Key 'B') - The "Time Travel" Demo

- Pre-requisite: Code configured to start at 19:59:50.
- Action: Unlock the door. Press 'B' to toggle Night Mode enabled.
- Observation: Watch the clock. When it hits 20:00, an 'N' appears.
- Test 1 (Silence): Lock and Unlock. Result: No Sound.
- Test 2 (Fast Lock): The Auto-lock timer starts at 2s (High Security) instead of 5s.

Step 5: Lockout (Timer Implementation)

- Action: Enter wrong PIN (9 9 9 9 #) three times.
- Result: "Error" jingle plays. LCD shows LOCKED OUT!.
- Observation: "System is unresponsive for 30s."
- Ending: Watch timer count to 0. System resets.

Step 6: Bluetooth Mode (Communication Interrupt)

- Action: Unlock door (0 0 0 0 #). Press 'D'.
- Observation: LCD shows BLUETOOTH MODE.
- Action (PC Terminal):
    - Type 'L' Door Locks (Red LED).
    - Type 'U' Door Unlocks (Green LED).
    - Type 'T' Door Toggles state.
- Action: Press '*' on the keypad to disconnect and return to normal mode.

Mode / Feature	Timer Concept Used	Interrupt Concept Used
Night Mode	**Time-of-Day Tracking:** Aggregating 10ms PIT ticks into Seconds, Minutes, and Hours to simulate an RTC (Real-Time Clock).	**Background Processing:** The clock updates and checks the 20:00-06:00 window inside the ISR, keeping the main loop free for UI tasks.
Lockout	**Time Delay Creation:** Counting down a specific long interval (30s) using a software counter driven by the hardware timer.	**Non-blocking Delay:** The ISR decrements the `lockout_ticks` counter in the background so the LCD continues to refresh the countdown.
Servo Lock	**Pulse Width Modulation (PWM):** Generating a precise 50Hz signal using FTM2 by matching the Counter (CNT) against a Channel Value (CnV).	**Hardware Automation:** The signal is generated entirely by the FTM hardware. The CPU only intervenes to change the duty cycle (position).
Buzzer (Audio)	**Digital Waveform Generation:** Generating variable frequency square waves using FTM3 to create specific musical notes (pitch).	**Hardware Automation:** Similar to the servo, the FTM3 module drives the output pin automatically, allowing complex jingles without blocking execution.

Keypad	N/A	**GPIO Interrupt (I/O Handling):** Detecting a "Falling Edge" on Port B/C to wake the CPU only when a button is physically pressed.
Bluetooth Mode	N/A	**Communication Interrupt:** The UART4 module triggers an interrupt (RDRF) immediately when data arrives, allowing for asynchronous remote control.
Permanent Unlock	**Timer State Control:** Explicitly disabling the counting mechanism (autolock_ticks = -1) to prevent the ISR from triggering a lock event.	**Conditional Logic in ISR:** The interrupt handler checks the state before decrementing, demonstrating control flow within an interrupt context.

Code:

```
#include "MK64F12.h"
#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>
#include <string.h>


// Configuration
#define LED_ACTIVE_HIGH 1

#if LED_ACTIVE_HIGH
  #define LED_ON(bit)  (GPIOD_PSOR = (1u << bit))
  #define LED_OFF(bit) (GPIOD_PCOR = (1u << bit))
#else
  #define LED_ON(bit)  (GPIOD_PCOR = (1u << bit))
  #define LED_OFF(bit) (GPIOD_PSOR = (1u << bit))
#endif

// Timing constants
```

```
#define PIN_LEN         4
#define PIT0_TICK_MS    10U

// Auto-lock times
#define AUTOLOCK_TICKS_DAY   (5000U / PIT0_TICK_MS)
#define AUTOLOCK_TICKS_NIGHT (2000U / PIT0_TICK_MS)

// Lockout time
#define LOCKOUT_TICKS        (30000U / PIT0_TICK_MS)
#define MAX_FAILED_ATTEMPTS  3

// Hardware Constants
#define FTM2_MOD_VALUE      3276U
#define SERVO_LOCKED_CNV    164U
#define SERVO_UNLOCKED_CNV  328U
#define FTM3_CLK_FREQ       1310720

// Globals

typedef enum {
    STATE_LOCKED = 0,
    STATE_UNLOCKED,
    STATE_PERM_UNLOCKED,
    STATE_CHANGE_OLD_PIN,
    STATE_CHANGE_NEW_PIN,
    STATE_LOCKOUT,
    STATE_BLUETOOTH_MODE
} LockState;

static char stored_pin[PIN_LEN] = { '1','2','3','4' };
static char input_buf[PIN_LEN];
static uint8_t input_len = 0;
static LockState lock_state = STATE_LOCKED;
static uint8_t failed_attempts = 0;

// Config Flags
static bool cfg_night_mode_enabled = false;

// Timer Globals
volatile int32_t autolock_ticks = -1;
volatile int32_t lockout_ticks = -1;
static int32_t last_display_seconds = -1;

// Clock Globals
```

```
volatile uint8_t rtc_hour = 19;
volatile uint8_t rtc_min  = 59;
volatile uint8_t rtc_sec  = 50;
volatile bool is_night_time = false;
static uint8_t pit_subsecond_count = 0;

// UI Flags
volatile char g_key_pressed = 0;
static uint8_t lcd_dirty = 1;

// Bluetooth Globals
volatile char g_bt_command = 0;
static bool bt_locked_state = false;

// Delay
static void delay_ms(uint32_t ms) {
    volatile uint32_t i;
    while (ms--) for (i = 0; i < 21000; i++) __asm("nop");
}

// Hardware Init
static void GPIO_Init(void) {
    SIM_SCGC5 |= SIM_SCGC5_PORTA_MASK | SIM_SCGC5_PORTB_MASK |
SIM_SCGC5_PORTC_MASK | SIM_SCGC5_PORTD_MASK | SIM_SCGC5_PORTE_MASK;

    // LEDs
    PORTD_PCR0 = PORT_PCR_MUX(1); PORTD_PCR1 = PORT_PCR_MUX(1);
PORTD_PCR2 = PORT_PCR_MUX(1);
    GPIOD_PDDR |= (1u<<0) | (1u<<1) | (1u<<2);
    LED_OFF(0); LED_OFF(1); LED_OFF(2);

    // LCD
    PORTC_PCR5 = PORT_PCR_MUX(1); PORTC_PCR1 = PORT_PCR_MUX(1);
    GPIOC_PDDR |= (1u<<5) | (1u<<1);
    PORTD_PCR4 = PORT_PCR_MUX(1); PORTD_PCR5 = PORT_PCR_MUX(1);
PORTD_PCR6 = PORT_PCR_MUX(1); PORTD_PCR7 = PORT_PCR_MUX(1);
    GPIOD_PDDR |= (1u<<4) | (1u<<5) | (1u<<6) | (1u<<7);

    // Keypad Rows
    PORTB_PCR9 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK |
PORT_PCR_IRQC(0xA);
    PORTC_PCR2 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK |
PORT_PCR_IRQC(0xA);
```

```
    PORTC_PCR3 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK |
PORT_PCR_IRQC(0xA);
    PORTC_PCR4 = PORT_PCR_MUX(1) | PORT_PCR_PE_MASK | PORT_PCR_PS_MASK |
PORT_PCR_IRQC(0xA);
    GPIOB_PDDR &= ~(1u<<9); GPIOC_PDDR &= ~((1u<<2) | (1u<<3) | (1u<<4));
    NVIC_EnableIRQ(PORTB_IRQn); NVIC_EnableIRQ(PORTC_IRQn);

    // Keypad Cols
    PORTA_PCR1=PORT_PCR_MUX(1); PORTB_PCR23=PORT_PCR_MUX(1);
PORTA_PCR2=PORT_PCR_MUX(1); PORTC_PCR0=PORT_PCR_MUX(1);
    GPIOA_PDDR |= (1u<<1)|(1u<<2); GPIOB_PDDR |= (1u<<23); GPIOC_PDDR |= (1u<<0);
    GPIOA_PCOR = (1u<<1)|(1u<<2); GPIOB_PCOR = (1u<<23); GPIOC_PCOR = (1u<<0);
}

// UART Bluetooth
static void UART4_Init(void) {
    SIM_SCGC1 |= SIM_SCGC1_UART4_MASK;
    PORTE_PCR24 = PORT_PCR_MUX(3); // TX
    PORTE_PCR25 = PORT_PCR_MUX(3); // RX
    UART4_C2 &= ~(UART_C2_TE_MASK | UART_C2_RE_MASK);
    uint16_t sbr = (uint16_t)(21000000 / (16 * 9600));
    UART4_BDH = (sbr >> 8) & 0x1F;
    UART4_BDL = (sbr & 0xFF);
    UART4_C2 |= UART_C2_RIE_MASK;
    NVIC_EnableIRQ(UART4_RX_TX_IRQn);
    UART4_C2 |= (UART_C2_TE_MASK | UART_C2_RE_MASK);
}

void UART4_RX_TX_IRQHandler(void) {
    if (UART4_S1 & UART_S1_RDRF_MASK) {
        char c = UART4_D;
        g_bt_command = c;
    }
}

// Outputs
static void Set_LEDs(bool locked, bool unlocked, bool error) {
    if(locked) LED_ON(0); else LED_OFF(0);
    if(unlocked) LED_ON(1); else LED_OFF(1);
    if(error) LED_ON(2); else LED_OFF(2);
}

static void Buzzer_Init(void) {
    SIM_SCGC3 |= SIM_SCGC3_FTM3_MASK; SIM_SCGC5 |= SIM_SCGC5_PORTD_MASK;
```

```
    PORTD_PCR3 = PORT_PCR_MUX(4); FTM3_MODE |= FTM_MODE_WPDIS_MASK;
    FTM3_SC=0; FTM3_CNT=0; FTM3_MOD=0; FTM3_C3SC = 0x28; FTM3_C3V=0;
    FTM3_SC = 0x0C;
}

static void Play_Tone(uint16_t freq, uint32_t ms) {
    if (cfg_night_mode_enabled && is_night_time) return;
    if (freq == 0) FTM3_C3V = 0;
    else { uint16_t mod = FTM3_CLK_FREQ / freq; FTM3_MOD = mod; FTM3_C3V = mod/2;
FTM3_CNT = 0; }
    delay_ms(ms); FTM3_C3V = 0; FTM3_MOD = 0;
}

// Jingles
void Jingle_Unlock() { Play_Tone(523,100); delay_ms(50); Play_Tone(659,100); delay_ms(50);
Play_Tone(784,200); }
void Jingle_Error()  { Play_Tone(150,300); delay_ms(100); Play_Tone(150,300); }
void Jingle_Beep()   { Play_Tone(2000,50); }
void Jingle_Lock()   { Play_Tone(784,100); delay_ms(50); Play_Tone(523,200); }

static void Servo_Init(void) {
    SIM_SCGC6|=SIM_SCGC6_FTM2_MASK; FTM2_MODE|=FTM_MODE_WPDIS_MASK;
    FTM2_MOD=FTM2_MOD_VALUE; FTM2_C0SC=0x28;
PORTB_PCR18=PORT_PCR_MUX(3);
    FTM2_SC=0x0F;
}
static void Servo_Move(bool open) { FTM2_C0V = open ? SERVO_UNLOCKED_CNV :
SERVO_LOCKED_CNV; }

// LCD Logic
static void LCD_SetRS(uint8_t h){if(h)GPIOC_PSOR=(1u<<5);else GPIOC_PCOR=(1u<<5);}
static void LCD_Pulse(void){GPIOC_PSOR=(1u<<1);for(int
i=0;i<800;i++);GPIOC_PCOR=(1u<<1);for(int i=0;i<800;i++);}
static void LCD_Nibble(uint8_t n){GPIOD_PCOR=(0xF<<4); if(n&1)GPIOD_PSOR=(1u<<4);
if(n&2)GPIOD_PSOR=(1u<<5); if(n&4)GPIOD_PSOR=(1u<<6);
if(n&8)GPIOD_PSOR=(1u<<7);}
static void LCD_Send(uint8_t v, uint8_t d){LCD_SetRS(d); LCD_Nibble(v>>4); LCD_Pulse();
LCD_Nibble(v&0xF); LCD_Pulse(); delay_ms(1);}
static void LCD_Init(void){delay_ms(20); LCD_Send(0x33,0); LCD_Send(0x32,0);
LCD_Send(0x28,0); LCD_Send(0x0C,0); LCD_Send(0x06,0); LCD_Send(0x01,0);
delay_ms(2);}
static void LCD_Pos(uint8_t r, uint8_t c){uint8_t a=(r==0)?0:0x40; LCD_Send(0x80|(a+c),0);}
static void LCD_Print(char *s){while(*s)LCD_Send(*s++,1);}
```

```
static void Update_Display(void) {
    char l1[17], l2[17];
    char time_s[10];
    char mode_char = ' ';
    if (is_night_time) mode_char = 'N';
    if (cfg_night_mode_enabled && is_night_time) mode_char = '!';

    sprintf(time_s, "%02d:%02d%c", rtc_hour, rtc_min, mode_char);

    switch(lock_state) {
        case STATE_LOCKED:
            snprintf(l1, 17, "LOCKED   %s", time_s);
            char mask[5];
            for(int i=0; i<4; i++) mask[i] = (i<input_len)?'*':'_';
            mask[4]=0;
            snprintf(l2, 17, "PIN: %-10s", mask);
            break;

        case STATE_UNLOCKED:
            snprintf(l1, 17, "UNLOCKED %s", time_s);
            if (autolock_ticks > 0) {
                int sec = (autolock_ticks + 99)/100;
                snprintf(l2, 17, "AUTO-LOCK: %ds  ", sec);
            } else {
                snprintf(l2, 17, "A:P B:N C:P D:BT");
            }
            break;

        case STATE_PERM_UNLOCKED:
            snprintf(l1, 17, "PERM UNLOCKED   ");
            snprintf(l2, 17, "PRESS * TO LOCK ");
            break;

        case STATE_LOCKOUT:
            snprintf(l1, 17, "LOCKED OUT!     ");
            snprintf(l2, 17, "WAIT: %ds      ", (int)((lockout_ticks+99)/100));
            break;

        case STATE_CHANGE_OLD_PIN:
            snprintf(l1, 17, "OLD PIN:        ");
            char omask[5]; for(int i=0;i<4;i++) omask[i]=(i<input_len)?input_buf[i]:'_'; omask[4]=0;
            snprintf(l2, 17, "%-16s", omask);
            break;
```

```
        case STATE_CHANGE_NEW_PIN:
            snprintf(l1, 17, "NEW PIN:        ");
            char nmask[5]; for(int i=0;i<4;i++) nmask[i]=(i<input_len)?input_buf[i]:'_'; nmask[4]=0;
            snprintf(l2, 17, "%-16s", nmask);
            break;

        case STATE_BLUETOOTH_MODE:
            snprintf(l1, 17, "BLUETOOTH MODE  ");
            if (bt_locked_state) snprintf(l2, 17, "STATUS: LOCKED  ");
            else                 snprintf(l2, 17, "STATUS: OPEN    ");
            break;

        default:
            snprintf(l1, 17, "ERROR           ");
            snprintf(l2, 17, "                ");
    }
    LCD_Pos(0,0); LCD_Print(l1);
    LCD_Pos(1,0); LCD_Print(l2);
}

// Logic
static void Set_State(LockState new_state) {
    lock_state = new_state;
    input_len = 0;
    lcd_dirty = 1;

    switch(lock_state) {
        case STATE_LOCKED:
            Set_LEDs(true, false, false); Servo_Move(false); autolock_ticks = -1;
            break;
        case STATE_UNLOCKED:
            Set_LEDs(false, true, false); Servo_Move(true);
            autolock_ticks = (is_night_time && cfg_night_mode_enabled) ?
AUTOLOCK_TICKS_NIGHT : AUTOLOCK_TICKS_DAY;
            break;
        case STATE_PERM_UNLOCKED:
            Set_LEDs(false, true, true); Servo_Move(true); autolock_ticks = -1;
            break;
        case STATE_LOCKOUT:
            Set_LEDs(false, false, true);
            break;
        case STATE_BLUETOOTH_MODE:
            bt_locked_state = false;
            Set_LEDs(false, true, true);
```

```
            autolock_ticks = -1;
            break;
        default:
            Set_LEDs(false, false, true);
            break;
    }
}

static void Handle_Input(char k) {
    if (lock_state == STATE_LOCKOUT) { Jingle_Error(); return; }
    Jingle_Beep();

    switch (lock_state) {
        case STATE_LOCKED:
            if(k>='0' && k<='9' && input_len<4) input_buf[input_len++] = k;
            else if(k=='*') input_len=0;
            else if(k=='#') {
                bool match = true; for(int i=0;i<4;i++) if(input_buf[i]!=stored_pin[i]) match=false;
                if(match) { failed_attempts=0; Jingle_Unlock(); Set_State(STATE_UNLOCKED); }
                else {
                    failed_attempts++; Set_LEDs(false,false,true); Jingle_Error(); delay_ms(500);
                    if(failed_attempts >= MAX_FAILED_ATTEMPTS) { lockout_ticks =
LOCKOUT_TICKS; Set_State(STATE_LOCKOUT); }
                    else Set_State(STATE_LOCKED);
                }
            }
            break;

        case STATE_UNLOCKED:
            if(k=='*') { Jingle_Lock(); Set_State(STATE_LOCKED); }
            else if(k=='A') { Set_State(STATE_PERM_UNLOCKED); Jingle_Unlock(); }
            else if(k=='B') { cfg_night_mode_enabled = !cfg_night_mode_enabled; lcd_dirty = 1;
Jingle_Unlock(); }
            else if(k=='C') { Set_State(STATE_CHANGE_OLD_PIN); }
            else if(k=='D') { Set_State(STATE_BLUETOOTH_MODE); }
            break;

        case STATE_PERM_UNLOCKED:
            if(k=='*') { Jingle_Lock(); Set_State(STATE_LOCKED); }
            break;

        case STATE_BLUETOOTH_MODE:
            if(k=='*') { Set_State(STATE_UNLOCKED); }
            break;
```

```
        case STATE_CHANGE_OLD_PIN:
            if(k>='0' && k<='9' && input_len<4) input_buf[input_len++] = k;
            else if(k=='*') Set_State(STATE_UNLOCKED);
            else if(k=='#') {
                bool match = true; for(int i=0;i<4;i++) if(input_buf[i]!=stored_pin[i]) match=false;
                if(match) { Jingle_Unlock(); Set_State(STATE_CHANGE_NEW_PIN); }
                else { Jingle_Error(); Set_State(STATE_UNLOCKED); }
            }
            break;

        case STATE_CHANGE_NEW_PIN:
            if(k>='0' && k<='9' && input_len<4) input_buf[input_len++] = k;
            else if(k=='*') Set_State(STATE_UNLOCKED);
            else if(k=='#') {
                if(input_len==4) { for(int i=0;i<4;i++) stored_pin[i]=input_buf[i]; Jingle_Unlock();
Set_State(STATE_LOCKED); }
            }
            break;

        default: break;
    }
    lcd_dirty = 1;
}

// Timer / ISR
static char ScanKey() {
    const char t[] = {'1','2','3','A','4','5','6','B','7','8','9','C','*','0','#','D'};
    for(int c=0;c<4;c++) {
        GPIOA_PSOR=(1u<<1)|(1u<<2); GPIOB_PSOR=(1u<<23); GPIOC_PSOR=(1u<<0);
        if(c==0)GPIOA_PCOR=(1u<<1); if(c==1)GPIOB_PCOR=(1u<<23);
if(c==2)GPIOA_PCOR=(1u<<2); if(c==3)GPIOC_PCOR=(1u<<0);
        for(int d=0;d<400;d++)__asm("nop");
        if(!(GPIOB_PDIR&(1u<<9))) return t[c]; if(!(GPIOC_PDIR&(1u<<2))) return t[4+c];
        if(!(GPIOC_PDIR&(1u<<3))) return t[8+c]; if(!(GPIOC_PDIR&(1u<<4))) return t[12+c];
    }
    return 0;
}
void PORTB_IRQHandler(void){ delay_ms(5); char k=ScanKey(); if(k)g_key_pressed=k;
PORTB_ISFR=0xFFFFFFFF; PORTC_ISFR=0xFFFFFFFF; GPIOA_PCOR=(1u<<1)|(1u<<2);
GPIOB_PCOR=(1u<<23); GPIOC_PCOR=(1u<<0); }
void PORTC_IRQHandler(void){ PORTB_IRQHandler(); }

static void PIT0_Init(void) {
```

```
    SIM->SCGC6 |= SIM_SCGC6_PIT_MASK; PIT->MCR &= ~PIT_MCR_MDIS_MASK;
PIT->MCR |= PIT_MCR_FRZ_MASK;
    PIT->CHANNEL[0].LDVAL = 210000 - 1; PIT->CHANNEL[0].TCTRL = 3;
NVIC_EnableIRQ(PIT0_IRQn);
    if (rtc_hour >= 20 || rtc_hour < 6) is_night_time = true;
}
void PIT0_IRQHandler(void) {
    PIT->CHANNEL[0].TFLG = 1;
    if (autolock_ticks > 0) autolock_ticks--;
    if (lockout_ticks > 0) lockout_ticks--;

    // Wall Clock
    pit_subsecond_count++;
    if (pit_subsecond_count >= 100) {
        pit_subsecond_count = 0;
        rtc_sec++;
        if (rtc_sec >= 60) {
            rtc_sec = 0; rtc_min++;
            if (rtc_min >= 60) {
                rtc_min = 0; rtc_hour++;
                if (rtc_hour >= 24) rtc_hour = 0;
            }
        }
        if (rtc_hour >= 20 || rtc_hour < 6) is_night_time = true;
        else is_night_time = false;
        lcd_dirty = 1;
    }
}

// Main
int main(void) {
lu    GPIO_Init(); LCD_Init(); Servo_Init(); Buzzer_Init(); PIT0_Init(); UART4_Init();
    Set_State(STATE_LOCKED); Jingle_Lock();

    while (1) {
        // Timers
        if (lock_state == STATE_UNLOCKED && autolock_ticks == 0) { Jingle_Lock();
Set_State(STATE_LOCKED); }
        if (lock_state == STATE_LOCKOUT && lockout_ticks == 0) { failed_attempts = 0;
Set_State(STATE_LOCKED); Jingle_Beep(); }

        // Bluetooth Logic
        if (g_bt_command != 0) {
            char c = g_bt_command;
```

```
            g_bt_command = 0;

            if (lock_state == STATE_BLUETOOTH_MODE) {
                if (c == 'u' || c == 'U') {
                    Jingle_Unlock(); Servo_Move(true); Set_LEDs(false, true, true);
                    bt_locked_state = false; lcd_dirty = 1;
                }
                else if (c == 'l' || c == 'L') {
                    Jingle_Lock(); Servo_Move(false); Set_LEDs(true, false, true);
                    bt_locked_state = true; lcd_dirty = 1;
                }
                else if (c == 't' || c == 'T') {
                    if (bt_locked_state) { Jingle_Unlock(); Servo_Move(true); Set_LEDs(false, true,
true); bt_locked_state = false; }
                    else { Jingle_Lock(); Servo_Move(false); Set_LEDs(true, false, true);
bt_locked_state = true; }
                    lcd_dirty = 1;
                }
            }
        }

        // Keypad Input
        if (g_key_pressed != 0) {
            char k = g_key_pressed; g_key_pressed = 0;
            Handle_Input(k);
        }

        // Update Display
        int32_t current_sec = -1;
        if (autolock_ticks > 0) current_sec = autolock_ticks / 100;
        else if (lockout_ticks > 0) current_sec = lockout_ticks / 100;

        if (lcd_dirty || (current_sec != -1 && current_sec != last_display_seconds)) {
            Update_Display();
            if(current_sec != -1) last_display_seconds = current_sec;
            lcd_dirty = 0;
        }
        delay_ms(10);
    }
}
```